INFORME PRÁCTICA 2
CALIBRACIÓN DE CÁMARA

Andrés Gil Vicente & Jorge Carnicero Príncipe

23 de octubre de 2025

Índice

1. Preparación del Entorno de Trabajo **4**

1.1. Carga de Librerías Necesarias 4

2. Apartado A: Calibración de Cámara (Derecha e Izquierda) **4**

2.1. Tarea A.1: Carga de Imágenes 5

2.2. Tarea A.2: Detección de Esquinas 6

2.2.1. Definición del Patrón de Calibración 6

2.2.2. Detección Inicial con findChessboardCorners 6

2.2.3. Refinamiento con cornerSubPix 7

2.3. Tarea A.3: Comprobación de Detecciones 7

2.3.1. Funciones Auxiliares de Visualización y Guardado 8

2.4. Tarea A.4: Obtención de Coordenadas 3D de las Esquinas 9

2.5. Tarea A.5: Calibración de la Cámara Izquierda 10

2.5.1. Resultados de calibración - Cámara Izquierda: 11

2.6. Pregunta A.1: Calibración de la Cámara Derecha 12

2.6.1. Comparación cámara derecha e izquierda 15

2.7. Pregunta A.2: Diferencias entre findChessboardCorners y cornerSubPix . . . 16

2.7.1. cv2.findChessboardCorners() 16

2.7.2. cv2.cornerSubPix() 16

2.7.3. Comparación Visual 17

2.8. Pregunta A.3: Número Mínimo de Imágenes para Calibración 19

2.8.1. Metodología Propuesta 20

2.8.2. Plan experimental riguroso y reproducible: 20

3. Apartado B: Corrección de Distorsión (Ojo de Pez) **21**

3.1. Tarea B.1: Carga de Imágenes 21

3.2. Tarea B.2: Detección de Esquinas 21

3.2.1. Verificación Visual 22

3.3. Tarea B.3: Obtención de Coordenadas del Tablero 23

3.4. Tarea B.4: Definición de Parámetros para Calibración 24
3.5. Tarea B.5: Calibración 24
3.6. Pregunta B.1: Corrección de Distorsión 25
3.6.1. Generación de Mapas de Remapeo 25
3.6.2. Aplicación de la Corrección 26
3.6.3. Visualización y Comparación 26
3.6.4. Resultados - Parámetros de calibración fisheye: 27
3.6.5. Análisis de Resultados 29

4. Conclusiones **30**
4.1. Calibración de Sistemas Estereoscópicos 30
4.2. Detección y Refinamiento de Esquinas 30
4.3. Calibración Fisheye 30

1. Preparación del Entorno de Trabajo

1.1. Carga de Librerías Necesarias

Para el desarrollo de esta práctica de calibración de cámara, es fundamental cargar las librerías que proporcionan las herramientas adecuadas para el procesamiento y análisis de imágenes. Estas son las siguientes:

Código 1.0: Importación de librerías básicas

```
import cv2
print("OpenCV should be 4.8.0.76 Current version:", cv2.__version__)
from typing import List
import numpy as np
import imageio
import copy
import glob
```

Las librerías utilizadas cumplen funciones específicas:

- **cv2**: Librería principal de OpenCV para procesamiento de imágenes y calibración de cámaras.
- **numpy**: Manipulación eficiente de arrays y matrices numéricas.
- **imageio**: Lectura y escritura de imágenes en diferentes formatos.
- **copy**: Creación de copias profundas de objetos para evitar modificaciones no deseadas.
- **glob**: Búsqueda de archivos mediante patrones.
- **typing**: Anotaciones de tipos para mejorar la legibilidad del código.

2. Apartado A: Calibración de Cámara (Derecha e Izquierda)

El objetivo de este apartado es obtener los parámetros intrínsecos y extrínsecos de las cámaras izquierda y derecha de un sistema estereoscópico. La calibración de cámaras es un proceso fundamental en visión por ordenador, ya que permite establecer correspondencias precisas entre puntos del mundo real (3D) y sus proyecciones en la imagen (2D).

Se trabajará con imágenes de las carpetas `left` y `right`, que contienen imágenes del patrón de calibración (tablero de ajedrez) en diferentes posiciones y orientaciones.

2.1. Tarea A.1: Carga de Imágenes

Se define el método `load_images()` para cargar imágenes desde el disco al espacio de memoria:

Código 2.0: Función para cargar múltiples imágenes

```
def load_images(filenames: List) -> List:
    return [imageio.imread(filename) for filename in filenames]
```

Esta función toma como argumento una lista de rutas de archivos y devuelve una lista de imágenes ya leídas y cargadas en memoria. Se utilizará para cargar todas las imágenes de calibración de forma masiva.

Código 2.1: Carga de imágenes de la cámara izquierda

```
# Patrón que cumplen todos los paths de los archivos de imágenes
imgs_gen_path = '../data/left/*.jpg'

# Cogemos todos los paths que cumplen ese patrón genérico y los ordenamos
imgs_path = sorted(glob.glob(imgs_gen_path))

# Cargamos las imágenes usando la función definida
imgs = load_images(imgs_path)
```

La función `glob.glob()` nos permite encontrar archivos que coincidan con un patrón específico, en este caso todos los archivos con extensión `.jpg` de la carpeta de imágenes `left`.

El resultado es una lista ordenada con 19 imágenes cargadas.

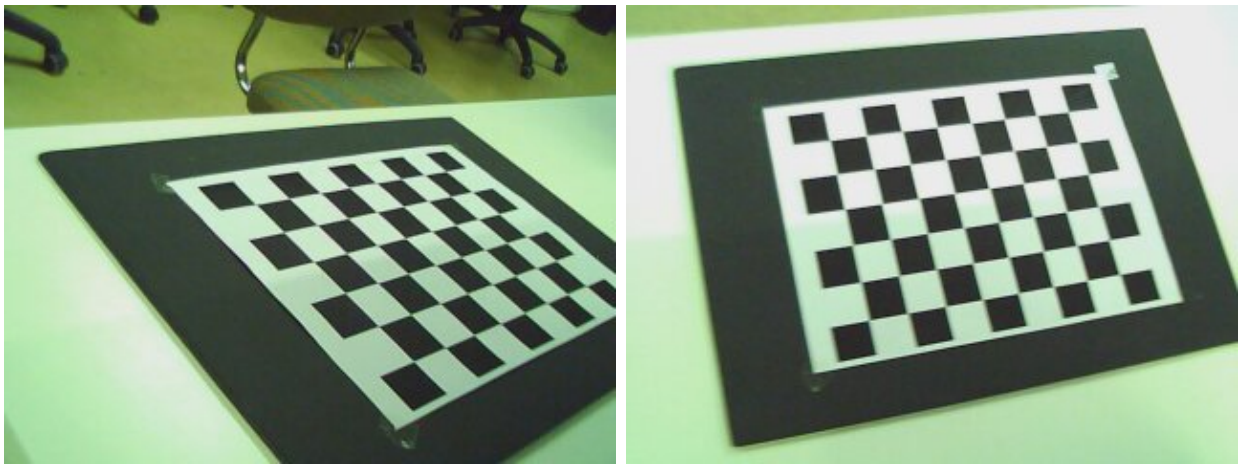


Figura 1: Imágenes 0 y 18 en la cámara izquierda cargadas correctamente

2.2. Tarea A.2: Detección de Esquinas

La detección de esquinas es un paso crucial en la calibración. Se utilizan dos funciones complementarias de OpenCV:

- `cv2.findChessboardCorners()`: para la detección inicial
- `cv2.cornerSubPix()`: para el refinamiento

2.2.1. Definición del Patrón de Calibración

En primer lugar debemos definir el tamaño del patrón de calibración que se busca detectar. Lo hacemos contando manualmente las esquinas interiores que hay en cada fila y columna del tablero de ajedrez:

Código 2.2: Definición del tamaño del patrón

```
# Formato (columnas, filas) de esquinas interiores
pattern_size = (8, 6)
```

2.2.2. Detección Inicial con findChessboardCorners

Código 2.3: Detección de esquinas con findChessboardCorners

```
# Detectar esquinas en todas las imágenes
corners = [cv2.findChessboardCorners(
    cv2.cvtColor(img, cv2.COLOR_RGB2GRAY), pattern_size)
    for img in imgs]
```

La función `cv2.findChessboardCorners()` devuelve una tupla con dos elementos:

1. Un booleano que indica si la detección fue exitosa (`True`) o no (`False`)
2. Un array con las coordenadas de las esquinas detectadas (válido solo si la detección fue exitosa)

Además, es importante saber que le debemos pasar como parámetros la imagen que contiene el patrón del tablero de ajedrez (conviene hacerlo en escala de grises), y las dimensiones del patrón, que es lo que definimos anteriormente contando manualmente.

El resultado para cada imagen, dejando aparte el booleano, tiene dimensiones $(48, 1, 2)$ porque hay $6 \times 8 = 48$ esquinas interiores, y cada esquina se describe mediante dos coordenadas: x e y.

2.2.3. Refinamiento con cornerSubPix

Para mejorar la precisión de las detecciones, se aplica `cv2.cornerSubPix()`:

Código 2.4: Refinamiento de esquinas con cornerSubPix

```
# Copia de seguridad de las esquinas detectadas
corners_copy = copy.deepcopy(corners)

# Criterio de parada del algoritmo iterativo
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.01)

# Convertir imágenes a escala de grises
imgs_gray = [cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) for img in imgs]


# Refinar detecciones
corners_refined = [cv2.cornerSubPix(i, cor[1], pattern_size, (-1, -1),
↪  criteria)
                   if cor[0] else []
                   for i, cor in zip(imgs_gray, corners_copy)]
```

El criterio de parada especifica que el algoritmo se detendrá cuando se alcance una precisión de 0.01 píxeles o se completen 30 iteraciones. Además, veamos cómo la función de cornerSubPix toma cómo argumento las esquinas que habíamos detectado antes con findChessboardCorners, ya que los usa como referencia inicial.

Diferencia entre findChessboardCorners y cornerSubPix:

- `findChessboardCorners()`: Realiza la detección inicial con precisión a nivel de píxel entero
- `cornerSubPix()`: Refina la posición utilizando gradientes de intensidad, logrando precisión subpíxel (coordenadas con decimales)

No obstante, a continuación entraremos más en detalle en esta explicación tal y como se nos pide en apartados posteriores de la práctica.

2.3. Tarea A.3: Comprobación de Detecciones

Para verificar que las detecciones son correctas, se utiliza `cv2.drawChessboardCorners()`:

Código 2.5: Visualización de esquinas detectadas

```
# Copia de seguridad para no perder las imágenes originales
imgs_copy = copy.deepcopy(imgs)

# Dibujar esquinas en cada imagen
for i in range(len(corners_copy)):
    ret = corners_copy[i][0]
    if ret:  # Solo dibujar si la detección fue exitosa
        cv2.drawChessboardCorners(imgs_copy[i], pattern_size,
                                  corners_refined[i], ret)
        cv2.imshow(f"Img {i} and corners", imgs_copy[i])
        cv2.waitKey(0)

cv2.destroyAllWindows()
```

La función `drawChessboardCorners()` modifica directamente la imagen pasada como parámetro, dibujando las esquinas detectadas y las conexiones entre ellas, todo ello encima del tablero que ya tenía la propia imagen. Por esta razón, es importante trabajar sobre una copia de seguridad.

2.3.1. Funciones Auxiliares de Visualización y Guardado

Para trabajar con mayor comodidad, se implementan funciones auxiliares para estandarizar la visualización y guardado de imágenes, que usaremos repetidamente durante la práctica :

Código 2.6: Funciones auxiliares

```
import os

def show_image(img: np.array, img_name: str = "Image"):
    cv2.imshow(img_name, img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

def write_image(output_folder: str, img_name: str, img: np.array):
    img_path = os.path.join(output_folder, img_name)
    os.makedirs(output_folder, exist_ok=True)
    cv2.imwrite(img_path, img)
```

Se guardan todas las imágenes con las esquinas detectadas:

Código 2.7: Guardado de resultados

```
for i in range(len(imgs_copy)):
    write_image("imagenes_esquinas_left_camera_cornerSubPix",
                f"patron_{i}.jpg", imgs_copy[i])
```

A continuación, vemos unos ejemplos de las imágenes resultado obtenidas de las esquinas de los patrones del tablero de ajedrez:

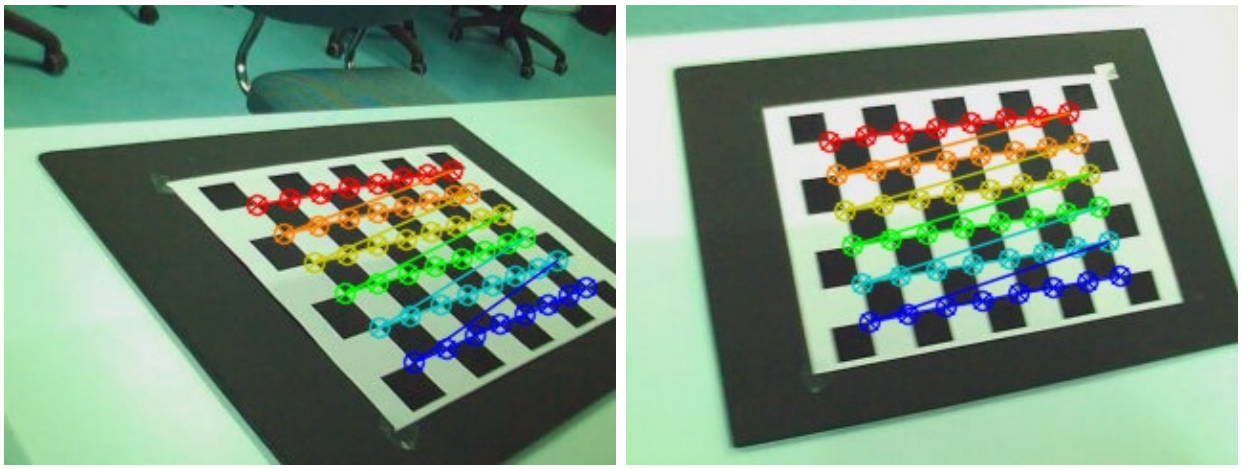


Figura 2: Esquinas detectadas de las imágenes 0 y 18 en la cámara izquierda

2.4. Tarea A.4: Obtención de Coordenadas 3D de las Esquinas

Para calibrar la cámara, es necesario conocer las coordenadas reales (3D) de las esquinas del patrón de calibración. Para ello, definimos la función `get_chessboard_points()`, que nos proporcionará dichas coordenadas de referencia:

Código 2.8: Función para obtener coordenadas 3D del tablero

```
def get_chessboard_points(chessboard_shape, dx, dy):

    # Separar columnas y filas
    cols, rows = chessboard_shape

    # Crear cuadrícula con coordenadas 2D del tablero
    grid_xy = np.mgrid[0:cols, 0:rows].T.reshape(-1, 2)

    # Crear matriz de puntos 3D
    objp = np.zeros([grid_xy.shape[0], 3], dtype=np.float32)

    # Rellenar coordenadas X e Y con distancias reales
    objp[:, 0] = grid_xy[:, 0] * dx  # Coordenada X
```

```
    objp[:, 1] = grid_xy[:, 1] * dy  # Coordenada Y
    # La coordenada Z permanece en 0 (tablero plano)

    return objp
```

Esta función genera las coordenadas exactas de las esquinas del tablero en el mundo real. El sistema de coordenadas está anclado al propio patrón, con el origen $[0, 0, 0]^T$ en la primera esquina detectada.

Código 2.9: Generación de puntos del tablero para todas las imágenes

```
dx, dy = 30, 30  # Tamaño de los cuadrados en mm

# Llamamos a la función que hemos definido
chessboard_points = [get_chessboard_points(pattern_size, dx, dy)
                     for _ in corners_refined]
```

El resultado tiene dimensiones $(48, 3)$ porque hay 48 esquinas (recordemos 8 por fila y 6 por columnas), cada una descrita por sus coordenadas (x, y, z), donde $z = 0$ siempre.

Ejemplo de salida:

```
[[  0.   0.   0.]
 [ 30.   0.   0.]
 [ 60.   0.   0.]
 ...
 [180. 150.   0.]
 [210. 150.   0.]]
```

2.5. Tarea A.5: Calibración de la Cámara Izquierda

Con todos los datos preparados, se procede a calibrar la cámara usando `cv2.calibrateCamera()`.

Para ello, primero filtramos los corners detectados, usando solo aquellos donde la detección fue correcta (lo marca el booleano que devolvía la función de detección)

Además, lo convertimos a array de numpy para trabajar correctamente con los datos:

Código 2.10: Filtrado de detecciones válidas

```
# Seleccionar solo las esquinas con detección exitosa
valid_corners = [cor[1] for cor in corners if cor[0]]

# Convertir a array de numpy
valid_corners = np.asarray(valid_corners, dtype=np.float32)
```

Código 2.11: Calibración de la cámara

```
# Obtener tamaño de las imágenes
img_size = (imgs_gray[0].shape[1], imgs_gray[0].shape[0])  # (320, 240)

# Calibrar la cámara
rms, intrinsics, dist_coeffs, rvecs, tvecs = cv2.calibrateCamera(
    chessboard_points, valid_corners, img_size, None, None)

# Obtener parámetros extrínsecos
extrinsics = list(map(lambda rvec, tvec: np.hstack((cv2.Rodrigues(rvec)[0],
↪  tvec)),
                      rvecs, tvecs))
```

La función `cv2.calibrateCamera()` devuelve:

- **rms**: Error cuadrático medio de reproyección
- **intrinsics**: Matriz de parámetros intrínsecos K (3×3)
- **dist_coeffs**: Coeficientes de distorsión
- **rvecs**: Vectores de rotación para cada imagen
- **tvecs**: Vectores de traslación para cada imagen

2.5.1. Resultados de calibración - Cámara Izquierda:

La función de calibración de la cámara, nos devuelve los siguientes parámetros:

```
Intrinsics:
 [[417.77315866   0.         156.33829983]
  [  0.         421.35480199 134.32880063]
  [  0.           0.           1.        ]]

Distortion coefficients:
 [[-2.13049815e-01  2.86830664e+00  5.90990527e-03
   -3.86453734e-03 -2.23696363e+01]]

Root mean squared reprojection error: 0.1846287136881559
```

2.6. Pregunta A.1: Calibración de la Cámara Derecha

De la misma forma que hemos trabajado para la cámara izquierda, se repite el proceso completo pero para la cámara derecha:

Omitimos las explicaciones de los pasos realizados y solo mostramos el código desarrollado para facilitar la comprensión y lectura de la memoria:

Código 2.12: Carga de imágenes de la cámara derecha

```
imgs_gen_path_right = '../data/right/*.jpg'
imgs_path_right = sorted(glob.glob(imgs_gen_path_right))
imgs_right = load_images(imgs_path_right)
```

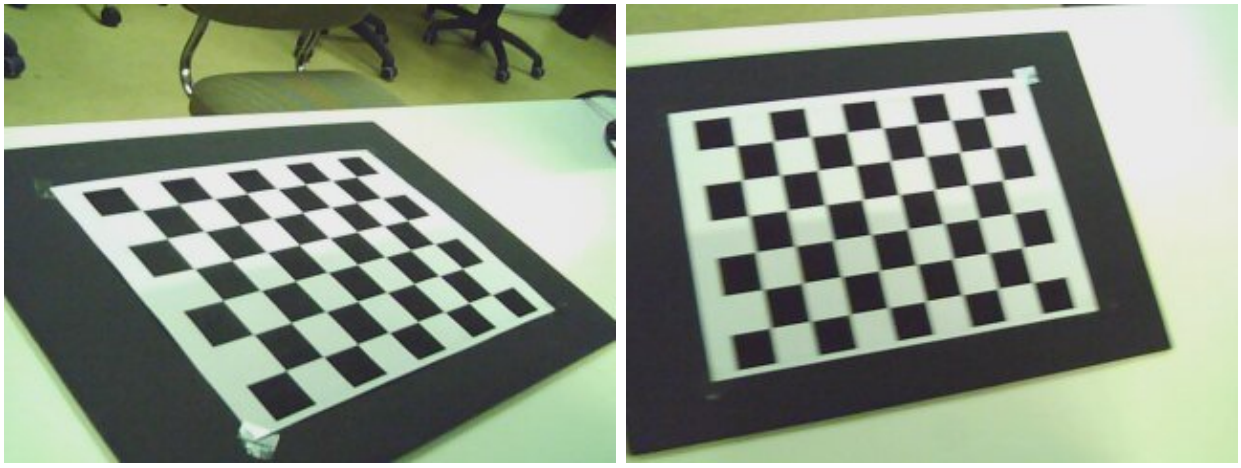


Figura 3: Imágenes 0 y 18 en la cámara derecha cargadas correctamente

Código 2.13: Detección y refinamiento de esquinas - Cámara derecha

```
# Detección inicial
corners_right = [cv2.findChessboardCorners(
    cv2.cvtColor(img, cv2.COLOR_RGB2GRAY), pattern_size)
    for img in imgs_right]

# Copia de seguridad
corners_copy_right = copy.deepcopy(corners_right)

# Criterio de parada
criteria_right = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30,
↪  0.01)

# Conversión a escala de grises
```

```
imgs_gray_right = [cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) for img in
↪  imgs_right]

# Refinamiento
corners_refined_right = [cv2.cornerSubPix(i, cor[1], pattern_size, (-1, -1),
↪  criteria_right)
                         if cor[0] else []
                         for i, cor in zip(imgs_gray_right,
                          ↪  corners_copy_right)]
```

Código 2.14: Visualización de esquinas - Cámara derecha

```
imgs_copy_right = copy.deepcopy(imgs_right)

for i in range(len(corners_copy_right)):
    ret = corners_copy_right[i][0]
    if ret:
        cv2.drawChessboardCorners(imgs_copy_right[i], pattern_size,
                                  corners_refined_right[i], ret)
        cv2.imshow(f"Img {i} and corners", imgs_copy_right[i])
        cv2.waitKey(0)

cv2.destroyAllWindows()
```

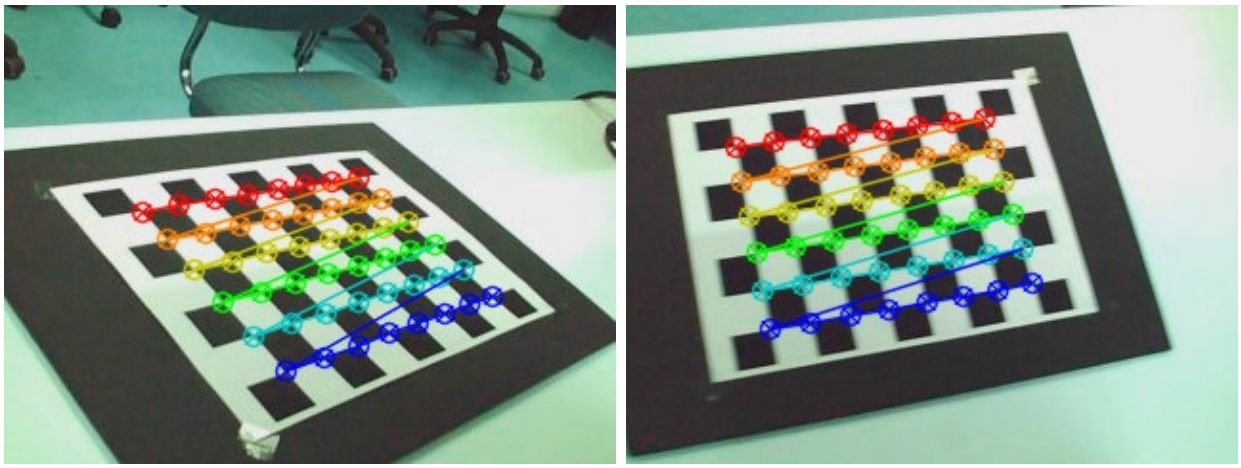


Figura 4: Esquinas detectadas de las imágenes 0 y 18 en la cámara derecha

Código 2.15: Obtención de coordenadas 3D - Cámara derecha

```
chessboard_points_right = [get_chessboard_points(pattern_size, dx, dy)
                           for _ in corners_refined_right]
```

Código 2.16: Filtrado de detecciones válidas - Cámara derecha

```
# Para que coincidan las dimensiones, filtramos las detecciones válidas
chessboard_points_valid = []
valid_corners_right = []

for i, found in enumerate(corners_right):
    if found[0]:  # Solo cogemos los que tienen True
        chessboard_points_valid.append(chessboard_points_right[i])
        valid_corners_right.append(corners_right[i][1])

# Convertir a array de numpy
valid_corners_right = np.asarray(valid_corners_right, dtype=np.float32)
```

Código 2.17: Calibración de la cámara derecha

```
img_size_right = (imgs_gray_right[0].shape[1], imgs_gray_right[0].shape[0])

rms_right, intrinsics_right, dist_coeffs_right, rvecs_right, tvecs_right = \
    cv2.calibrateCamera(chessboard_points_valid, valid_corners_right,
                        img_size_right, None, None)

# Obtener parámetros extrínsecos
extrinsics_right = list(map(lambda rvec, tvec:
↪   np.hstack((cv2.Rodrigues(rvec)[0], tvec)),
                            rvecs_right, tvecs_right))
```

Resultados de calibración - Cámara Derecha:

```
Intrinsics:
 [[431.11974701   0.         143.80060613]
  [  0.         431.81832564 135.91696325]
  [  0.           0.           1.        ]]

Distortion coefficients:
 [[-0.11858684  0.00164086  0.00744943 -0.00649521  0.17890046]]

Root mean squared reprojection error: 0.14569098303833106

Extrinsics:
 [array([[ 7.18794566e-01,  6.11836355e-01, -3.30137314e-01,
        -8.79978025e+01],
       [-1.75226243e-01,  6.18972963e-01,  7.65616245e-01,
        -3.47436595e+01],
       [ 6.72777925e-01, -4.92472075e-01,  5.52124188e-01,
```

```
          5.25578055e+02]]),
          array([[ 7.30987556e-01,  5.79483361e-01, -3.60355695e-01,
         -1.25892650e+02],
        [-1.95864673e-01,  6.84030487e-01,  7.02665869e-01,
         -5.96394948e+01],
        [ 6.53677461e-01, -4.43059056e-01,  6.13518093e-01,
          5.06187400e+02]]),
          array([[ 7.52837521e-01,  5.53466664e-01, -3.56244745e-01,
         -1.18966693e+02], [-2.00115364e-01,  7.08078376e-01,  6.77184506e-01,
...
        [-1.55553101e-01,  9.33587846e-01,  3.22826528e-01,
         -9.33560923e+01],
        [ 2.54073723e-01, -2.77997708e-01,  9.26371318e-01,
          6.43822721e+02]])]
```

Se puede observar que la cámara derecha tiene un error de reproyección ligeramente menor (0.146 vs 0.185) y coeficientes de distorsión más moderados que la cámara izquierda.

2.6.1. Comparación cámara derecha e izquierda

Comparamos visualmente el resultado de la búsqueda de esquinas con la cámara derecha, frente a los obtenidos con la cámara izquierda:

Código 2.18: Comparación resultados ambas cámaras

```
# Cogemos una imagen cualquiera de ejemplo
n = 3

# Cámara izquierda
cv2.imshow("Img pattern with left camera ",imgs_copy[n])
cv2.waitKey(0)
cv2.destroyAllWindows()

# Cámara derecha
cv2.imshow("Img pattern with right camera ",imgs_copy_right[n])
cv2.waitKey(0)
cv2.destroyAllWindows()
```

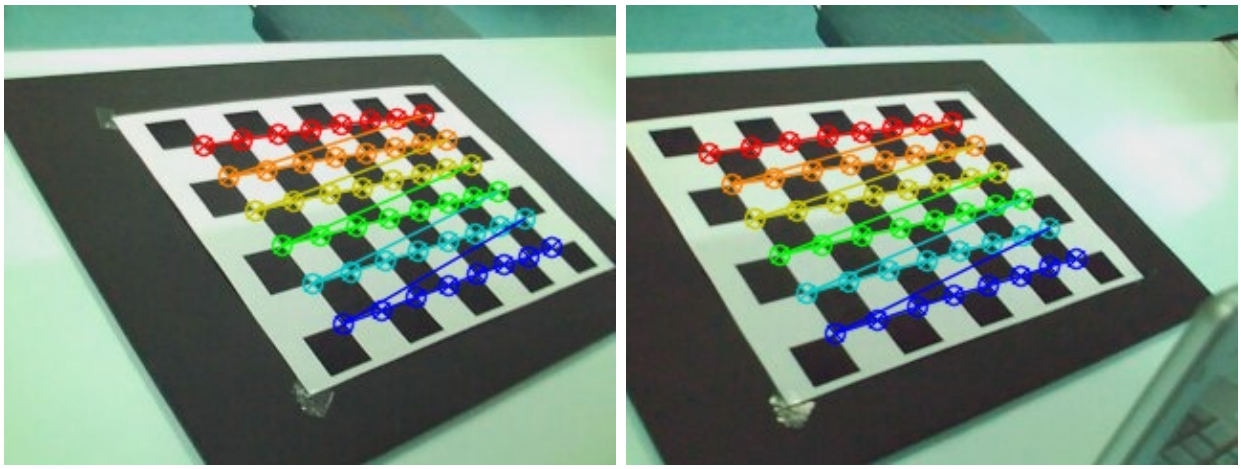


Figura 5: Comparación esquinas detectadas imagen 3 entre left y right

2.7. Pregunta A.2: Diferencias entre findChessboardCorners y cornerSubPix

La calibración de una cámara requiere establecer correspondencias precisas entre puntos del mundo real (3D) y sus proyecciones en la imagen (2D). Para ello, se emplean patrones conocidos como por ejemplo, tableros de ajedrez, cuyas esquinas han de detectarse en las imágenes.

OpenCV proporciona dos funciones complementarias en este proceso: `cv2.cornerSubPix()` y `cv2.findChessboardCorners()`.

2.7.1. cv2.findChessboardCorners()

Esta función sirve para la **detección inicial de esquinas a nivel de píxel**. Realiza la detección automática de las esquinas interiores del patrón de calibración, devolviendo coordenadas con precisión a nivel de un píxel entero.

El resultado es una estimación inicial de la posición de cada punto característico (esquina interior), suficiente para validar la detección del patrón, pero no lo bastante precisa para una calibración de alta exactitud.

2.7.2. cv2.cornerSubPix()

Una vez detectadas las esquinas, se aplica `cv2.cornerSubPix()` para **refinar su posición** aprovechando la información del gradiente de la imagen en una vecindad alrededor de cada punto. El término *subpíxel* hace referencia a que esta función puede estimar la posición de la esquina con precisión superior a la resolución del píxel, es decir, con coordenadas que incluyen decimales. Por ejemplo, (46.3154 , 60.82371) en lugar de (46.33627, 60.791836).

El algoritmo evalúa iterativamente los cambios de intensidad (gradientes) dentro de una pequeña ventana centrada en cada esquina, y va ajustando la posición hacia el punto donde el gradiente se anula en ambas direcciones, lo que indica el centro exacto de la intersección de los bordes.

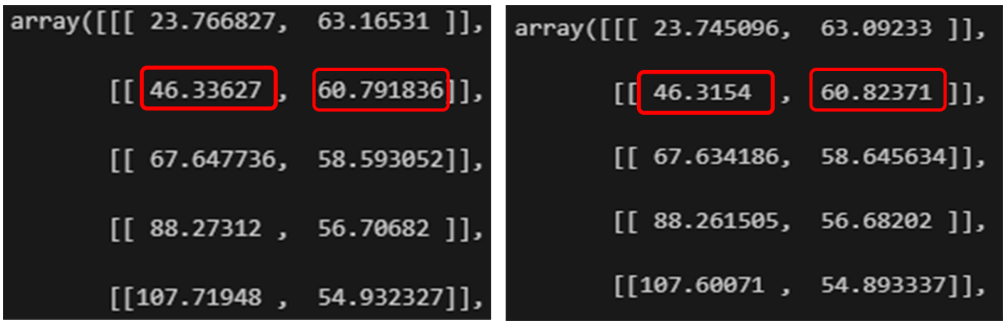


Figura 6: Coordenadas de los corners en findChessboardCorners() vs cornerSubPix() en la imagen 10

2.7.3. Comparación Visual

Para comparar ambos métodos de manera gráfica, generamos las imágenes con las detecciones de las esquinas que nos proporciona cada una de las funciones:

Código 2.19: Comparación visual de detecciones

```
imgs_copy_right_normal = copy.deepcopy(imgs_right)
imgs_copy_right_precise = copy.deepcopy(imgs_right)

for i in range(len(corners_copy_right)):
    ret = corners_copy_right[i][0]
    if ret:
        # Esquinas con findChessboardCorners
        cv2.drawChessboardCorners(imgs_copy_right_normal[i], pattern_size,
                                  corners_copy_right[i][1], ret)
        cv2.imshow(f"Img {i} - findChessboardCorners",
        ↪  imgs_copy_right_normal[i])
        cv2.waitKey(0)

        # Esquinas con cornerSubPix
        cv2.drawChessboardCorners(imgs_copy_right_precise[i], pattern_size,
                                  corners_refined_right[i], ret)
        cv2.imshow(f"Img {i} - cornerSubPix", imgs_copy_right_precise[i])
        cv2.waitKey(0)

cv2.destroyAllWindows()
```

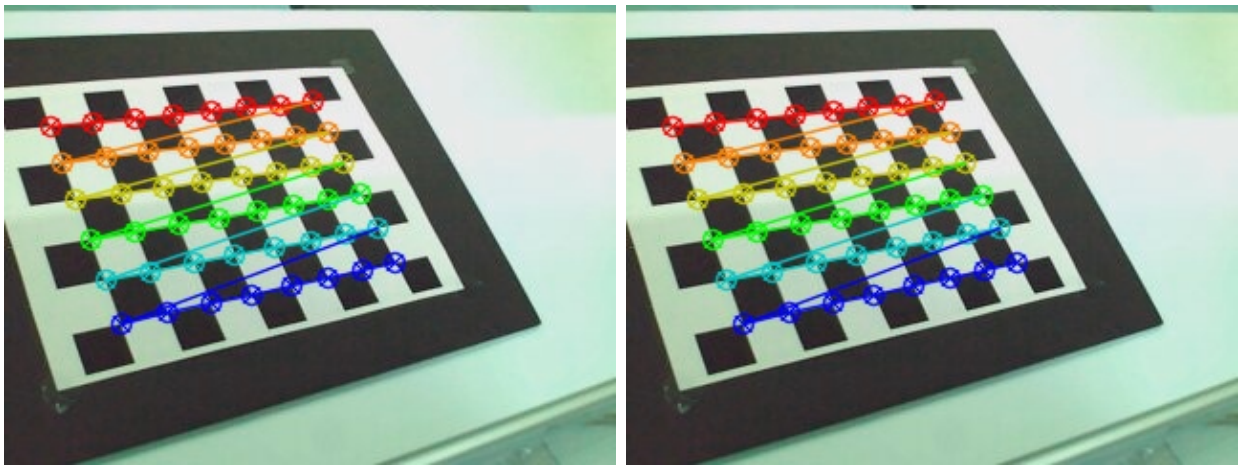


Figura 7: Comparación visual findChessboardCorners() vs cornerSubPix() en la imagen 10

Las diferencias visualmente son casi inapreciables, ya que las diferencias de coordenadas entre ambas funciones son inferiores a un píxel (del orden de décimas de píxel), demasiado pequeñas para ser detectadas a simple vista. Sin embargo, esta mejora en precisión es crucial para obtener una calibración exacta.

Con el objetivo de ilustrar y apoyar las diferencias que estamos viendo, hemos decidido aportar un par de gráficos adicionales que muestren las diferencias entre las coordenadas que encuentra una función y otra para las esquinas del patrón del tablero de ajedrez. Estas diferencias son pequeñas pero existentes:

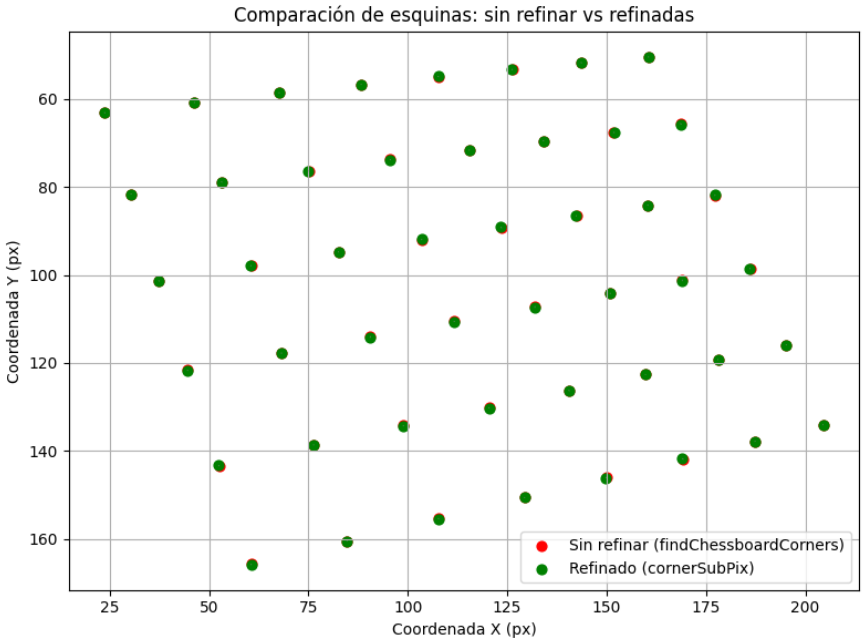


Figura 8: Gráfico de corners en findChessboardCorners() vs cornerSubPix() en la imagen 10

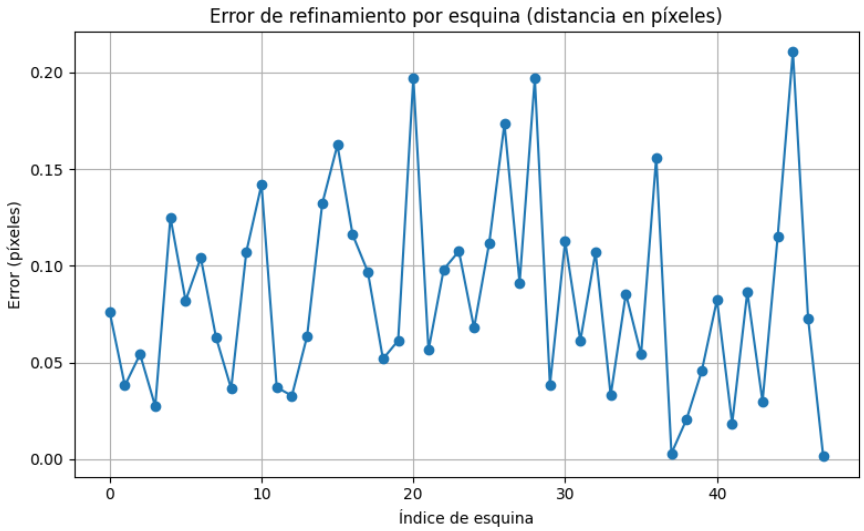


Figura 9: Gráfico de errores que se han corregido al pasar de findChessboardCorners() a cornerSubPix()

2.8. Pregunta A.3: Número Mínimo de Imágenes para Calibración

Un aspecto fundamental en la calibración de cámaras es determinar el número óptimo de imágenes necesarias para obtener resultados confiables. Para responder a esta pregunta, se propone un análisis mediante un diagrama de Pareto que relacione el número de imágenes utilizadas con el error de calibración.

A continuación mostramos el diagrama de Pareto que hemos obtenido:

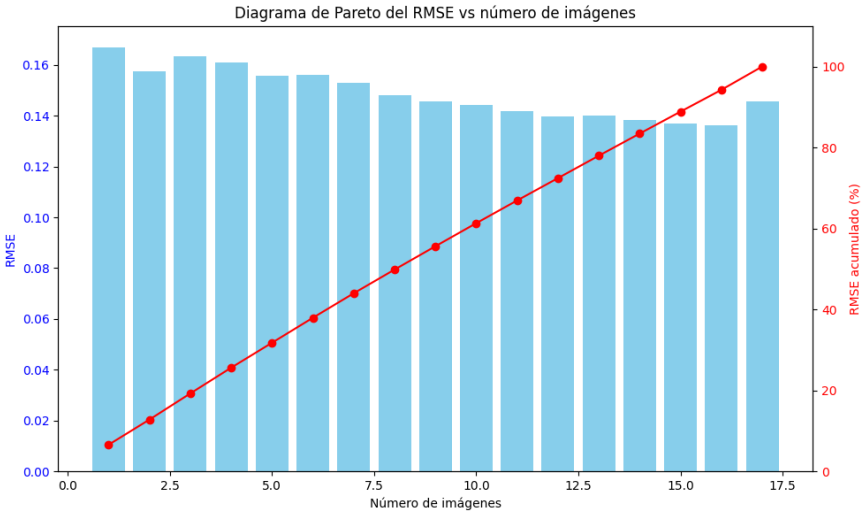


Figura 10: Gráfico de pareto, relación RMSE con cantidad de imágenes usadas

2.8.1. Metodología Propuesta

Además del diagrama de Pareto, hemos decidido representar otro gráfico adicional para apoyar la decisión de cuántas imágenes son necesarias para calibrar correctamente una cámara.

A continuación detallamos el proceso que hemos llevado a cabo.

2.8.2. Plan experimental riguroso y reproducible:

En nuestro caso, hemos utilizado las 18 imágenes disponibles de la cámara derecha.

El procedimiento consistió en realizar calibraciones incrementales: primero utilizando únicamente una imagen, luego dos, y así sucesivamente hasta incluir todas las imágenes del conjunto. Para cada calibración, se calculó el error cuadrático medio (RMSE) resultante, lo que permitió evaluar la precisión de la calibración en función del número de imágenes empleadas.

Este enfoque sistemático nos proporciona una visión clara de cómo varía el error al aumentar la cantidad de imágenes, facilitando la identificación del número óptimo de imágenes necesario para obtener una calibración confiable y consistente.

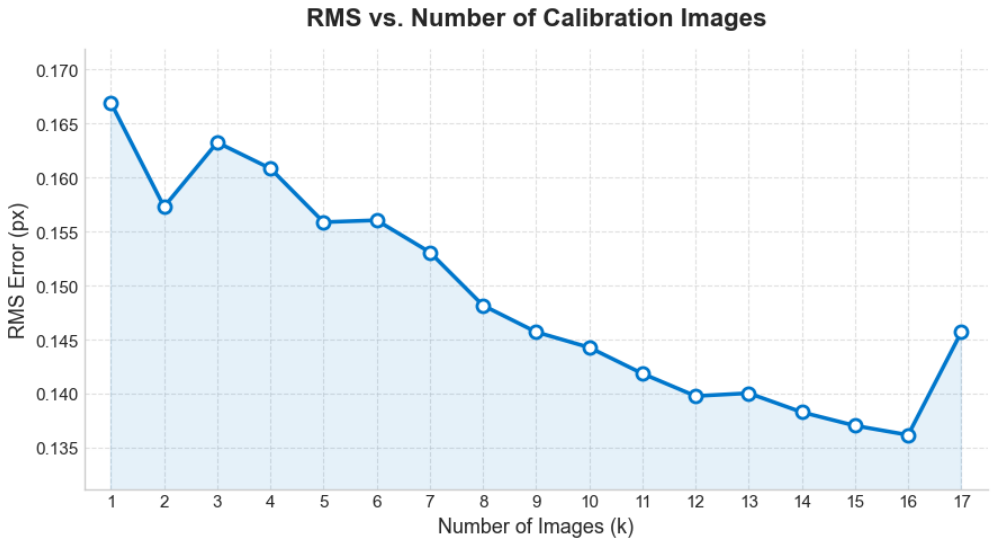


Figura 11: Gráfico de líneas que relaciona el nº de imágenes con el RMSE de calibración

A la conclusión que hemos llegado, observando los gráficos (Figura 10 y 11), es que el número óptimo de imágenes que se necesitan para calibrar con precisión la cámara, se sitúa en torno a 12 imágenes, ya que hasta ese punto se aprecia una reducción notable del RMSE, mientras que a partir de ahí la pendiente de disminución se reduce y las mejoras adicionales son menos significativas. Es decir, que es el punto idóneo para dejar de aportar imágenes ya que no nos reducen apenas el error.

3. Apartado B: Corrección de Distorsión (Ojo de Pez)

En este apartado se trabajará en la corrección de la distorsión producida por lentes de ojo de pez (fisheye). Este tipo de lentes introducen una distorsión radial significativa que deforma las imágenes, especialmente en las regiones periféricas. El objetivo es calibrar una cámara con lente ojo de pez y posteriormente utilizar los parámetros obtenidos para corregir la distorsión de las imágenes.

3.1. Tarea B.1: Carga de Imágenes

Se reutiliza el método `load_images()` para cargar las imágenes de la carpeta `fisheye`:

Código 3.0: Carga de imágenes fisheye

```
fisheye_imgs_path = '../data/fisheye/*.jpg'
imgs_fisheye_path = sorted(glob.glob(fisheye_imgs_path))
fisheye_imgs = load_images(imgs_fisheye_path)
```

El conjunto contiene 10 imágenes del patrón de calibración capturadas con una cámara con lente ojo de pez.

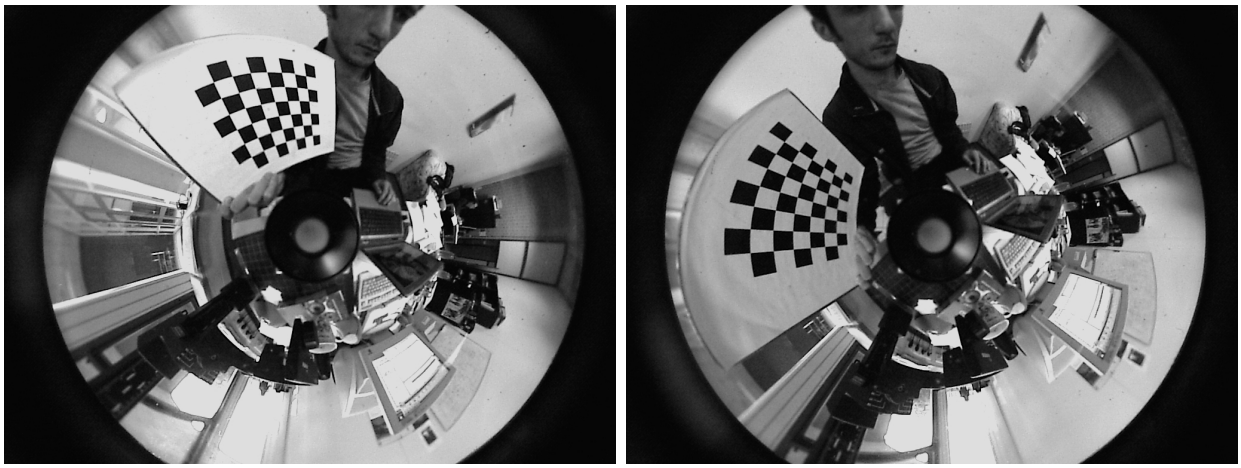


Figura 12: Carga correcta de las imágenes 3 y 4 con el efecto de ojo de pez

3.2. Tarea B.2: Detección de Esquinas

Se procede a detectar las esquinas del patrón en las imágenes fisheye:

Código 3.1: Definición del patrón y detección de esquinas

```
# Definir tamaño del patrón (contado manualmente)
fisheye_size_pattern = (7, 6)  # 7 columnas × 6 filas de esquinas interiores

# Inicializar lista para guardar corners
imgs_corners = []

# Parámetros para cv2.cornerSubPix()
subpix_criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30,
↪  0.1)

# Detección y refinamiento
for img in fisheye_imgs:
    # Detección inicial
    corners = cv2.findChessboardCorners(
        cv2.cvtColor(img, cv2.COLOR_RGB2GRAY), fisheye_size_pattern)

    # Convertir a escala de grises para cornerSubPix
    gray_img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)

    if corners[0]:  # Solo procesar si la detección fue exitosa
        refined_corners = cv2.cornerSubPix(gray_img, corners[1], (3, 3),
                                           (-1, -1), subpix_criteria)
        imgs_corners.append(refined_corners)
```

El resultado tiene dimensiones $(42, 1, 2)$ porque hay $7 \times 6 = 42$ esquinas interiores.

3.2.1. Verificación Visual

Código 3.2: Visualización de esquinas detectadas

```
fisheye_imgs_copy = copy.deepcopy(fisheye_imgs)

for i in range(len(imgs_corners)):
    cv2.drawChessboardCorners(fisheye_imgs_copy[i], fisheye_size_pattern,
                              imgs_corners[i], True)
    cv2.imshow(f"Img fisheye {i}", fisheye_imgs_copy[i])
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

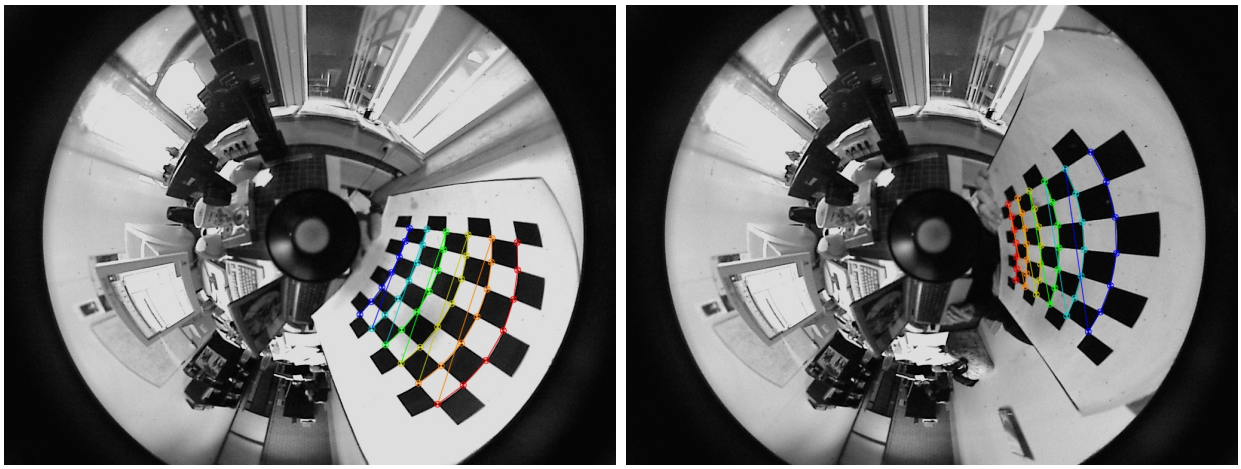


Figura 13: Esquinas detectadas de las imágenes con ojo de pez 0 y 1

3.3. Tarea B.3: Obtención de Coordenadas del Tablero

Se reutiliza la función `get_chessboard_points()` adaptándola al nuevo patrón:

Código 3.3: Obtención de coordenadas 3D del tablero fisheye

```
# Definir dimensiones y tamaño de cuadrados
chessboard_dims = (7, 6)  # Esquinas interiores
dx, dy = 30, 30  # Longitud de cuadrados en mm

# Generar puntos 3D para cada imagen con detección exitosa
# Reshape necesario para formato (n, 1, 3) requerido por
↪  cv2.fisheye.calibrate
fisheye_chessboard_points = [get_chessboard_points(chessboard_dims, dx,
↪  dy).reshape(-1, 1, 3)
                             for _ in imgs_corners]
```

El formato $(42, 1, 3)$ es específicamente requerido por la función `cv2.fisheye.calibrate()`.

Ejemplo de salida:

```
[[[  0.   0.   0.]]
 [[ 30.   0.   0.]]
 [[ 60.   0.   0.]]
 ...
 [[150. 150.   0.]]
 [[180. 150.   0.]]]
```

3.4. Tarea B.4: Definición de Parámetros para Calibración

Antes de calibrar, es necesario inicializar las variables que serán rellenadas por el algoritmo:

Código 3.4: Preparación de parámetros de calibración

```
# Flags de configuración del método de calibración
calibration_flags = cv2.fisheye.CALIB_RECOMPUTE_EXTRINSIC + \
                    cv2.fisheye.CALIB_FIX_SKEW

# Matriz de intrínsecos (3×3) - será rellenada por calibrateCamera
intrinsics = np.zeros((3, 3))

# Coeficientes de distorsión (4×1) - modelo fisheye usa 4 parámetros
distortion = np.zeros((4, 1))

# Vectores de rotación (uno por imagen)
rotations = [np.zeros((1, 1, 3), dtype=np.float64) for _ in imgs_corners]

# Vectores de traslación (uno por imagen)
traslations = [np.zeros((1, 1, 3), dtype=np.float64) for _ in imgs_corners]
```

Explicación de los parámetros:

- **calibration_flags**: Controlan el comportamiento del algoritmo
  - `CALIB_RECOMPUTE_EXTRINSIC`: Recalcula parámetros extrínsecos en cada iteración
  - `CALIB_FIX_SKEW`: Fija el parámetro de sesgo (skew) a cero
- **intrinsics**: Matriz K con parámetros intrínsecos (f_x, f_y, c_x, c_y)
- **distortion**: Vector con 4 coeficientes de distorsión radial específicos del modelo fisheye
- **rotations/traslations**: Describen la posición y orientación del patrón respecto a la cámara

3.5. Tarea B.5: Calibración

Se ejecuta el proceso de calibración utilizando el modelo fisheye de OpenCV:

Código 3.5: Calibración de cámara fisheye

```
# Calibrar cámara fisheye
rms, _, _, _, _ = cv2.fisheye.calibrate(
    fisheye_chessboard_points,      # Coordenadas 3D del mundo real
    imgs_corners,                   # Coordenadas 2D detectadas en imágenes
    gray_img.shape[::-1],           # Dimensiones de la imagen
```

```
    intrinsics,                    # Matriz de intrínsecos (output)
    distortion,                    # Coeficientes de distorsión (output)
    rotations,                     # Vectores de rotación (output)
    traslations,                   # Vectores de traslación (output)
    calibration_flags,             # Flags de configuración
    subpix_criteria                # Criterio de parada iterativo
)
```

La función `cv2.fisheye.calibrate()` estima los parámetros intrínsecos y de distorsión específicos del modelo de lente ojo de pez, a partir del conjunto de imágenes del patrón de calibración.

Resultados de calibración - Cámara Fisheye:

```
Intrinsic matrix:
 [[188.86914026   0.         503.92900347]
  [  0.         185.32638678 373.58044468]
  [  0.           0.           1.        ]]

Distortion coefficients:
 [[ 0.07734388]
  [ 0.01482361]
  [ 0.02351869]
  [-0.02757565]]
```

Obsérvese que el modelo fisheye utiliza 4 coeficientes de distorsión, a diferencia del modelo pinhole estándar que puede usar 5 o más parámetros.

3.6. Pregunta B.1: Corrección de Distorsión

Una vez calibrada la cámara fisheye, se dispone de la matriz de intrínsecos y los coeficientes de distorsión necesarios para eliminar la distorsión de las imágenes capturadas.

3.6.1. Generación de Mapas de Remapeo

La corrección de distorsión se realiza en dos pasos: primero se calculan los mapas de remapeo, luego se aplican a las imágenes.

Código 3.6: Cálculo de mapas de remapeo

```
# Dimensiones de las imágenes a corregir
dim = (fisheye_imgs[0].shape[0], fisheye_imgs[0].shape[1])  # (768, 1024)

# Generar mapas de remapeo
```

```
map1, map2 = cv2.fisheye.initUndistortRectifyMap(
    intrinsics,          # Matriz de intrínsecos
    distortion,          # Coeficientes de distorsión
    np.eye(3),           # Matriz de rectificación (identidad)
    intrinsics,          # Nueva matriz de cámara (misma que original)
    dim,                 # Dimensiones de salida
    cv2.CV_16SC2         # Tipo de dato de los mapas
)
```

Explicación de los mapas de remapeo:

- **map1**: Contiene, para cada píxel de la imagen rectificada, las coordenadas (x, y) del píxel correspondiente en la imagen distorsionada
- **map2**: Proporciona información de precisión subpíxel para mejorar la interpolación

Los mapas tienen dimensiones $(1024, 768)$ donde cada posición (i, j) indica de qué ubicación de la imagen original se debe tomar el valor del píxel.

3.6.2. Aplicación de la Corrección

Código 3.7: Corrección de distorsión usando cv2.remap()

```
# Copia de seguridad
undistorted_fisheye_imgs_copy = copy.deepcopy(fisheye_imgs)

# Aplicar corrección de distorsión a todas las imágenes
undistorted_fisheye_imgs = [cv2.remap(img, map1, map2,
                                      interpolation=cv2.INTER_LINEAR)
                            for img in undistorted_fisheye_imgs_copy]
```

La función `cv2.remap()` crea una nueva imagen tomando los valores de la imagen original según las coordenadas especificadas en `map1` y `map2`. La interpolación lineal (`INTER_LINEAR`) proporciona un buen equilibrio entre calidad y velocidad de procesamiento.

3.6.3. Visualización y Comparación

Código 3.8: Comparación de imágenes distorsionadas y corregidas

```
for i in range(len(undistorted_fisheye_imgs_copy)):
    # Mostrar ambas imágenes para comparar
    cv2.imshow(f"Original (Distorsionada) {i}", fisheye_imgs[i])
    cv2.imshow(f"Rectificada (Sin distorsión) {i}",
    ↪  undistorted_fisheye_imgs[i])
```

```
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

Código 3.9: Visualización de las dos primeras imágenes corregidas

```
# Mostrar las 2 primeras imágenes corregidas (según enunciado)
n = 0

cv2.imshow(f"Rectificada (Sin distorsión) {n}", undistorted_fisheye_imgs[n])
cv2.waitKey(0)
cv2.destroyAllWindows()

cv2.imshow(f"Rectificada (Sin distorsión) {n+1}",
↪  undistorted_fisheye_imgs[n+1])
cv2.waitKey(0)
cv2.destroyAllWindows()
```

3.6.4. Resultados - Parámetros de calibración fisheye:

```
Matriz de intrínsecos:
 [[188.86914026   0.         503.92900347]
  [  0.         185.32638678 373.58044468]
  [  0.           0.           1.        ]]

Coeficientes de distorsión:
 [[ 0.07734388]
  [ 0.01482361]
  [ 0.02351869]
  [-0.02757565]]
```



Figura 14: Primeras dos imágenes de ojo de pez sin distorsión

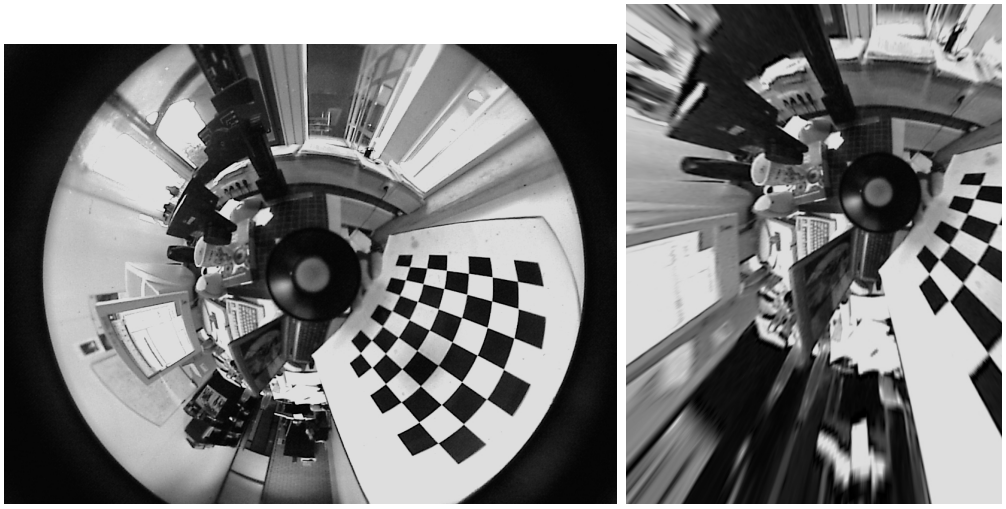


Figura 15: Primera imagen con distorsión comparada con sin distorsión

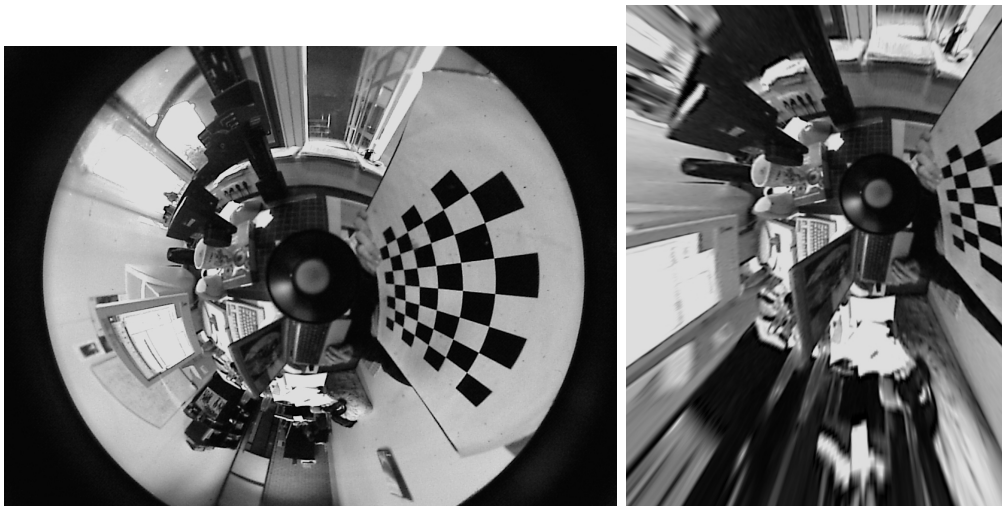


Figura 16: Segunda imagen con distorsión comparada con sin distorsión

3.6.5. Análisis de Resultados

La corrección de distorsión fisheye es evidente al comparar las imágenes originales con las rectificadas:

- **Líneas rectas**: En la imagen original, las líneas rectas del tablero aparecen curvadas, especialmente cerca de los bordes. Tras la corrección, estas líneas se vuelven algo más rectas.
- **Proporciones**: Los cuadrados del tablero mantienen proporciones más consistentes en toda la imagen corregida, mientras que en la original se deformaban hacia los bordes.
- **Áreas periféricas**: La distorsión más pronunciada se observaba en las regiones exteriores de la imagen, donde el efecto "ojo de pez" era más notorio. La corrección elimina este efecto.
- **Campo de visión**: La corrección puede resultar en un ligero recorte o "estiramiento" de las regiones periféricas para mantener la geometría euclidiana correcta.

4. Conclusiones

En esta práctica se han abordado los fundamentos de la calibración de cámaras y corrección de distorsión utilizando OpenCV. Los principales aprendizajes que hemos adquirido incluyen:

4.1. Calibración de Sistemas Estereoscópicos

Se ha completado exitosamente la calibración de ambas cámaras (izquierda y derecha) de un sistema estereoscópico:

- **Cámara izquierda**: RMS = 0.185
- **Cámara derecha**: RMS = 0.146

Ambos valores de error de reproyección son aceptables para aplicaciones de visión por ordenador, indicando una calibración precisa.

La cámara derecha mostró un error ligeramente menor, posiblemente debido a mejor calidad en las detecciones de esquinas o menor distorsión intrínseca.

4.2. Detección y Refinamiento de Esquinas

Se ha demostrado la importancia de utilizar tanto `findChessboardCorners()` como `cornerSubPix()`:

- La detección inicial proporciona una localización aproximada a nivel de píxel.
- El refinamiento subpíxel mejora la precisión mediante análisis de gradientes.
- Aunque las diferencias visuales son imperceptibles, el refinamiento es crucial para la exactitud de la calibración.

4.3. Calibración Fisheye

La calibración y corrección de distorsión de lentes ojo de pez ha demostrado:

- La corrección mediante mapas de remapeo es eficiente y efectiva.
- Las líneas rectas del mundo real se recuperan correctamente tras la rectificación.
- El método es aplicable a sistemas de objetivos de gran angular con distorsión radial significativa.